Scenario Based Question

1. You are working in a cybersecurity company developing a high-speed firewall. Network packets are processed as strings consisting only of the symbols `0` and `1`. A packet is considered valid only if it starts with `1`, the total number of `1`s seen so far is divisible by `3`, and the packet never ends with `00`. If an invalid pattern is detected, the packet is immediately rejected and the system moves to a dead state. While designing the automaton, you may consider an initial state, states that track the remainder of the count of `1`s, a state that remembers a trailing `0`, one or more accepting states, and a trap state. Design a deterministic finite automaton for this firewall validation system.

2. You are part of an embedded systems team designing a smart washing machine. Machine operations are represented by the symbols `F`, `W`, `R`, and `S`. The normal operation follows `F → W → R → S`. If a power failure occurs, the machine must resume from the last completed operation. If a leakage sensor signal `L` occurs at any time, the system must immediately move to a safe termination state `T`. While designing the automaton, you may consider a start state, operation states, a safety state, and a completion state. Design a deterministic finite automaton for this system.

3. You are working in an EdTech company developing an online examination platform. Student actions are represented using `L`, `Q`, `A`, and `S`. A student may navigate and submit answers multiple times, but `S` is allowed only after at least one `A` has occurred. Network interruptions represented by `D` may occur, after which the session resumes from the last valid state. While designing the automaton, you may consider states for logged-in users, active answering, submission-eligible state, and completed sessions. Design a deterministic finite automaton for valid exam sessions.

4. You are part of a smart city project designing a traffic signal controller. Traffic lights are represented by `R`, `G`, and `Y`. Normally, the signal cycles `R → G → Y → R`. A manual override signal `M` can force the light to `R` from any state, and an emergency signal `E` can force the light to `G`. After handling these events, the system resumes normal cycling. While designing the automaton, you may consider normal cycle states and override transitions. Design a deterministic finite automaton for this controller.

5. You are part of a compiler design team implementing lexical analysis. Identifiers consist of lowercase letters `a–z`, digits `0–9`, and underscore `_`. An identifier may optionally start with `_`, but it must contain at least one letter. While designing the automaton, you may consider a start state, states tracking whether a letter has been seen, looping states for valid continuations, and accepting states. Design a nondeterministic finite automaton for recognizing valid identifiers.

6. You are working in a system security team analyzing logs represented using `L`, `O`, `G`, `I`, `N`, `U`, `T`, and `X`. The sequence `LOGIN` represents login, `LOGOUT` represents logout, and `X` represents unrelated events. Whenever a `LOGIN` occurs, a corresponding `LOGOUT` must occur later, with any number of `X` symbols in between. Multiple logins may be pending simultaneously. While designing the automaton, you may consider states for partial word detection, active sessions, and completion. Design a nondeterministic finite automaton for this analysis.

7. A research group processes experimental data streams made up only of the symbols `A` and `B`. A stream is considered significant if it contains `AABB` or `ABAB` as a subsequence, meaning symbols appear in order but not necessarily consecutively. While designing the automaton, you may consider parallel tracking states for different partial matches and a shared accepting state. Design a nondeterministic finite automaton for this task.

8. You are designing an automated approval system where events are represented using `R`, `J`, and `A`. A request may be rejected multiple times but must eventually be approved. While designing the automaton, you may consider a start state, an active request state, looping rejection transitions, and an accepting approval state. Design a nondeterministic finite automaton for this workflow.

9. You are working in a network protocol design team. Session behavior is represented using `S`, `E`, and `R`. A valid session must start with `S` and eventually end with `E`. The symbol `R` may occur at any time and forces the protocol back to the initial state. While designing the automaton, you may consider start, active, reset, and accepting states. Design a nondeterministic finite automaton for this protocol.

10. You are analyzing sensor outputs represented using `H` and `L`. A sensor is considered stable only if two consecutive `H` symbols occur. If an `L` occurs after a single `H`, detection must restart. While designing the automaton, you may consider a start state, a partial detection state, reset transitions, and an accepting stability state. Design a nondeterministic finite automaton for this monitor.

11. You are part of a compiler development team implementing syntax checking. Expressions consist of parentheses `(` and `)` along with other irrelevant symbols. The system must accept only those strings in which parentheses are properly balanced and nested. While designing the automaton, you may consider stack operations for pushing `(` and popping `(` on `)`, along with acceptance based on empty stack and input. Design a pushdown automaton for this validation.

12. You are designing an XML validation tool. Documents contain opening tags `<a>` and closing tags `</a>`, `<b>` and `</b>`, and so on. Tags must be properly nested and closed in the correct order. While designing the automaton, you may consider stack symbols for

open tags, matching transitions, and acceptance based on empty stack. Design a pushdown automaton for XML validation.

13. You are designing a programming language runtime where function calls are represented by `C` and returns by `R`. Exception events are represented by `E` and may cause multiple active function calls to be exited at once. While designing the automaton, you may consider stack symbols for function calls and transitions that pop multiple symbols on `E`. Design a pushdown automaton for this behavior.

14. You are part of a database engine team designing a query parser. Queries use `(` and `)` to represent nested subqueries. If a subquery is invalid, it is discarded and parsing continues at the outer level. While designing the automaton, you may consider stack usage for nested queries and acceptance based on correct nesting. Design a pushdown automaton for this parser.

15. You are designing a vending machine where inputs are `5`, `10`, and `B`. Outputs include `D` and `C`. The output is produced immediately on the transition when sufficient money is inserted and `B` is pressed. While designing the machine, you may consider states representing accumulated credit and outputs associated with transitions. Design a Mealy machine for this vending system.

16. You are working on an automated toll booth. Inputs include `V` and `P`. Outputs include `O` and `A`. The output is generated immediately when payment is confirmed after vehicle detection. While designing the machine, you may consider waiting, payment, and open-gate states. Design a Mealy machine for this toll booth controller.

17. You are designing an elevator controller. Inputs include `1`, `2`, `3`, and `C`. Outputs include `M` and `D`. Since outputs depend on both the current state and the input received, the system is modeled as a Mealy machine. Design a Mealy machine for this elevator controller.

18. You are designing a digital lock where inputs are digits `0–9` and `R`. Outputs include `L`, `E`, and `U`. The displayed output depends only on the current state. While designing the machine, you may consider states for partial code entry and final unlocked state. Design a Moore machine for this digital lock.

19. You are designing a washing machine indicator panel. Inputs are `F`, `W`, `R`, and `S`, and outputs displayed are `FILL`, `WASH`, `RINSE`, and `SPIN`. The displayed output depends only on the current state. While designing the machine, you may consider one state per operation. Design a Moore machine for this indicator system.

20. You are designing a network connection status monitor. Inputs include `C`, `A`, and `F`. Outputs displayed are `DISCONNECTED`, `CONNECTING`, `CONNECTED`, and `ERROR`, depending only on the current state. While designing the machine, you may consider states corresponding to each status. Design a Moore machine for this monitor.